

Chapter 1

Basic Setting

1.1 The Abelian Sandpile Model

The Abelian sandpile model is a cellular automaton introduced by Bak, Tang and Wiesenfeld. A d -dimensional lattice holds a non-negative integer $h(x)$ at each site x , the number of grains. A site is *stable* when $h(x) < T$, where T is the toppling threshold; it is *unstable* when $h(x) \geq T$. An unstable site *topples*: it loses T grains and sends one grain to each of its $2d$ nearest neighbours. Grains that leave the boundary are lost (open boundary condition). Repeated toppling eventually returns every site to a stable configuration; this final configuration is independent of the order of topplings (the Abelian property).

In two dimensions the grid is $N \times N$, the threshold is $T = 4$, and each site has four orthogonal neighbours. In three dimensions the cube is $n \times n \times n$, the threshold is $T = 6$, and each site has six neighbours $(\pm x, \pm y, \pm z)$.

1.2 Implementation of the Sandpile

The model is implemented in Python with NumPy. Two classes, `Sandpile2D` and `Sandpile3D`, share the same structure; we describe the 2D case for brevity. The grid is stored as a NumPy integer array. Stabilisation does not rescan the whole grid each step; instead it keeps a queue of currently unstable sites, so the running time is $O(N^2 + \text{number of topples})$. The core routine is:

Listing 1.1: Stabilisation by queue (2D, simplified).

```

1 def stabilize(grid, N, T=4):
2     queue = deque(sites where grid >= T)
3     topples = 0
4     while queue:
5         (x, y) = queue.popleft()
6         if grid[x, y] < T:           # already stabilised by a neighbour
7             continue
8         grid[x, y] -= T              # topple
9         topples += 1
10        for (nx, ny) in neighbours(x, y): # up, down, left, right
11            if inside(nx, ny, N):
12                grid[nx, ny] += 1
13                if grid[nx, ny] >= T:
14                    queue.append((nx, ny))
15    return grid, topples

```

Each toppling event removes T grains from a site and distributes one grain to every in-bounds neighbour; any grain crossing the boundary is discarded. The 3D version is identical except that the threshold is 6 and each toppling distributes to six neighbours instead of four.

1.3 Experiment Design

The experiments follow the instructions given in the email correspondence that initiated this project. The protocol for a single trial is:

1. Start with an $N \times N$ (2D) or $n \times n \times n$ (3D) grid.

2. Put a fixed *background* of grains on every site: 3 grains in 2D, 5 grains in 3D. The background is chosen one below the threshold, so the grid is stable but on the verge of toppling.
3. Select three random distinct sites and add one extra grain to each. At least one site now reaches the threshold and an avalanche begins.
4. Stabilise the grid with the routine of Section 1.2.
5. Save the final stable grid to disk and record statistics: total topplings, number of distinct toppled sites, the grain distribution, the seed, and the perturbed positions.

This trial is repeated many times (10 000 in 2D, 2 000 in 3D). Each trial uses an independent random seed $s_i = s_0 + i$, so the perturbation sites differ every trial while the experiment as a whole remains reproducible. A central configuration file (`config.py`) holds all parameters—grid size, threshold, background, number of perturbation sites, number of trials, seed and output directories—so that the whole experiment can be retuned from one place. The driver (`main.py`) reads these parameters, runs the batch, and saves grids and aggregate statistics incrementally so that progress survives an interruption.

To speed things up, the experiment uses multiple threads (Python’s `concurrent.futures`). Each trial runs independently in its own thread, so trials do not interfere with one another and the total runtime is much shorter.

After the batch completes, the same driver produces the classification study of Chapter ??: for each direction (n, m) with $0 \leq n, m \leq 7$ it extracts the height pattern far from the centre along that direction and arranges the resulting images in a table.

1.4 Visualisation

The module `visualize.py` (built on Matplotlib) provides interactive viewers for exploring the saved grids. Each grain count is mapped to a fixed colour via a discrete 7-colour colormap (off-white $\rightarrow$ yellow $\rightarrow$ blue $\rightarrow$ green $\rightarrow$ orange $\rightarrow$ brown $\rightarrow$ red), so that the height pattern is read directly as a coloured image.

Two viewers are available: one for 2D grids and one for 3D cubes (browsed layer by layer). The 2D viewer can zoom into a small 18×18 neighbourhood around the cursor and save annotated snapshots—this was the main way we inspected the directional patterns in Chapter ?. Usage details are in the project’s `README.md`.